

Chapter 7 –

Design and Implementation

Design and implementation

- Software design and implementation is the stage in the software engineering process at which an executable software system is developed.
- Software design and implementation activities are invariably inter-leaved.
 - Software design is a creative activity in which you identify software components and their relationships, based on a customer's requirements.
 - Implementation is the process of realizing the design as a program.

Build or buy

- In a wide range of domains, it is now possible to buy off-the-shelf systems (COTS) that can be adapted and tailored to the users' requirements.
 - For example, if you want to implement a medical records system, you can buy a package that is already used in hospitals. It can be cheaper and faster to use this approach rather than developing a system in a conventional programming language.
- When you develop an application in this way, the design process becomes concerned with how to use the configuration features of that system to deliver the system requirements.

7.1 An object-oriented design process

- Structured object-oriented design processes involve developing a number of different system models.
- They require a lot of effort for development and maintenance of these models and, for small systems, this may not be cost-effective.
- However, for large systems developed by different groups design models are an important communication mechanism.

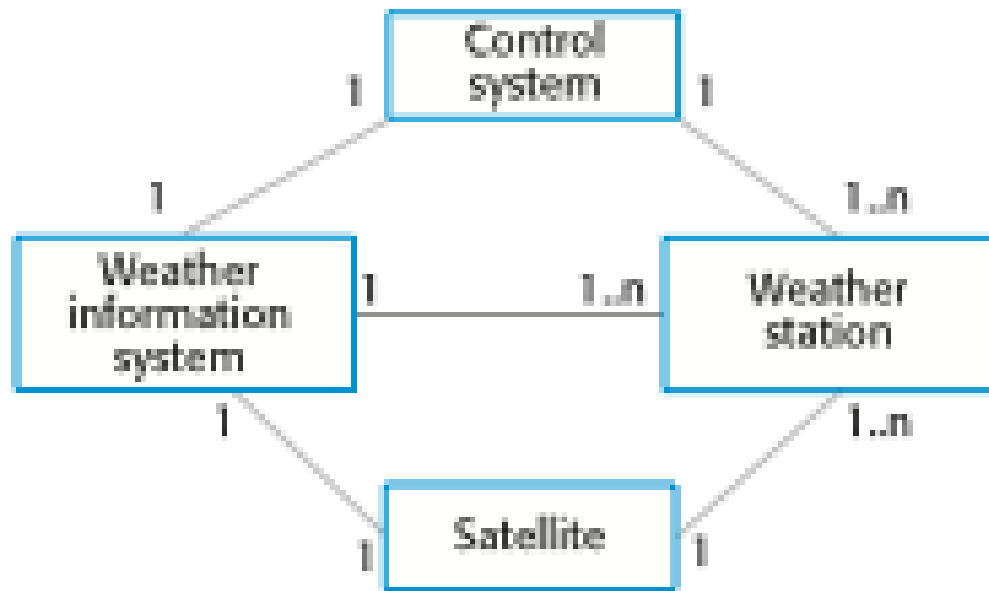
7.1 Process stages

- There are a variety of different object-oriented design processes that depend on the organization using the process.
- Common activities in these processes include:
 1. Define the context and modes of use of the system;
 2. Design the system architecture;
 3. Identify the principal system objects;
 4. Develop design models;
 5. Specify object interfaces.
- Process illustrated here using a design for a wilderness weather station.

7.1.1. System context and interactions

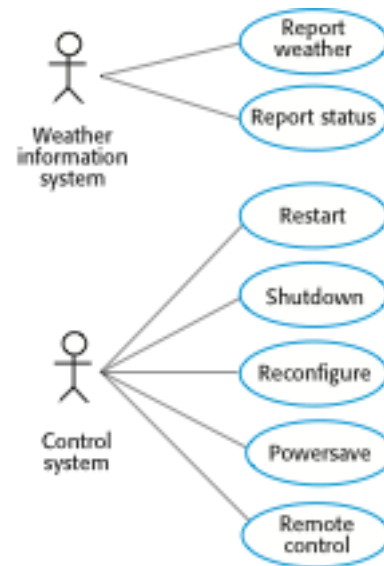
- Understanding the relationships between the software that is being designed and its external environment is essential for deciding how to provide the required system functionality and how to structure the system to communicate with its environment.
- Understanding of the context also lets you establish the boundaries of the system. Setting the system boundaries helps you decide what features are implemented in the system being designed and what features are in other associated systems.
- A system context model is a structural model that demonstrates the other systems in the environment of the system being developed.
- An interaction model is a dynamic model that shows how the system interacts with its environment as it is used.

7.1.1. System context for the weather station (continued)



The systems in the environment of each weather station are a weather information system, an onboard satellite system, and a control system.

7.1.1. Weather station use cases (continued)



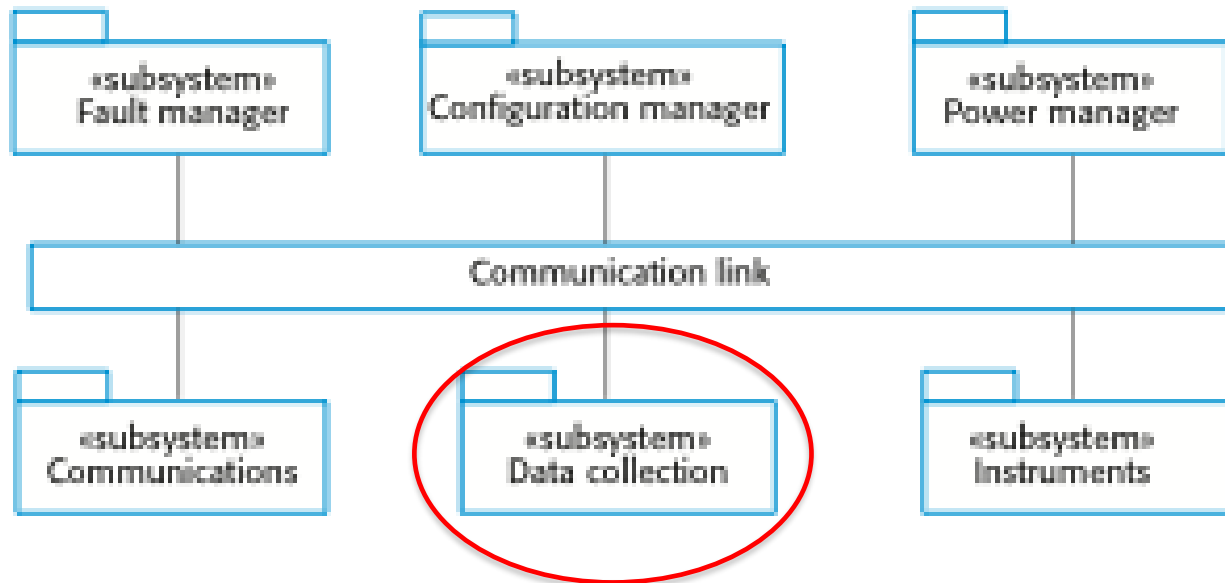
7.1.1. Use case description—Report weather (continued)

System	Weather station
Use case	Report weather
Actors	Weather information system, Weather station
Description	<u>The weather station sends a summary of the weather data</u> that has been collected from the instruments in the collection period to the weather information system. The data sent are the maximum, minimum, and average ground and air temperatures; the maximum, minimum, and average air pressures; the maximum, minimum, and average wind speeds; the total rainfall; and the wind direction as sampled at five-minute intervals.
Stimulus	The weather information system establishes a satellite communication link with the weather station and requests transmission of the data.
Response	The summarized data is sent to the weather information system.
Comments	Weather stations are usually asked to report once per hour but this frequency may differ from one station to another and may be modified in the future.

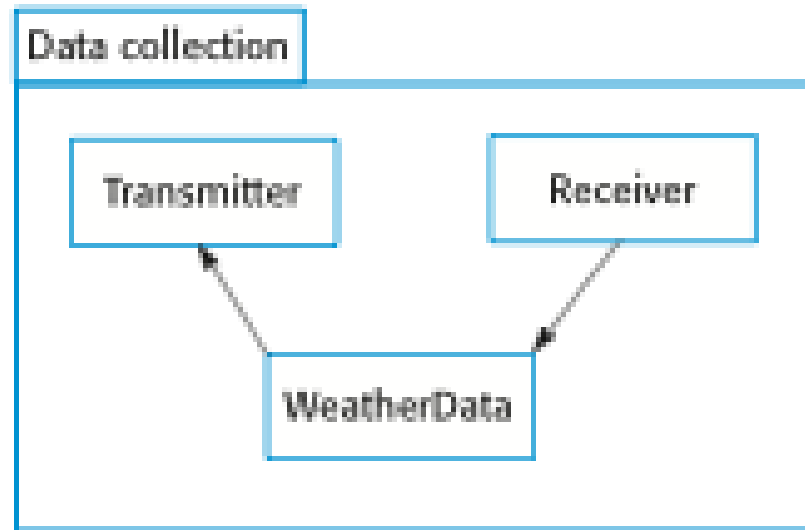
7.1.2. Architectural design

- Once interactions between the system and its environment have been understood, you use this information for designing the system architecture.
- You identify the major components that make up the system and their interactions, and then may organize the components using an architectural pattern such as a layered or client-server model.
- The weather station is composed of independent subsystems that communicate by broadcasting messages on a common infrastructure.

7.1.2. High-level architecture of the weather station (continued)



7.1.2. Architecture of data collection system (continued)



7.1.3. Object class identification (continued)

- Identifying object classes is often a difficult part of object oriented design.
- There is no 'magic formula' for object identification. It relies on the skill, experience and domain knowledge of system designers.
- Object identification is an iterative process. You are unlikely to get it right first time.
- Use a grammatical approach based on a natural language description of the system. Objects and attributes are nouns, and operations and services are verbs.
- Base the identification on tangible things in the application domain such as aircraft.
- Use a scenario-based analysis. The objects, attributes and methods in each scenario are identified.

7.1.3 Weather station description (continued)

A weather station is a package of software controlled instruments which collects data, performs some data processing and transmits this data for further processing. The instruments include air and ground thermometers, an anemometer, a wind vane, a barometer and a rain gauge. Data is collected periodically.

When a command is issued to transmit the weather data, the weather station processes and summarises the collected data. The summarised data is transmitted to the mapping computer when a request is received.

7.1.3 Weather station object classes (continued)

- Object class identification in the weather station system may be based on the tangible hardware and data in the system:
 - Ground thermometer, Anemometer, Barometer
 - *Application domain objects that are ‘hardware’ objects related to the instruments in the system.*
 - Weather station
 - *The basic interface of the weather station to its environment. It therefore reflects the interactions identified in the use-case model.*
 - Weather data
 - *Encapsulates the summarized data from the instruments.*

7.1.3 Weather station object classes (continued)

WeatherStation	WeatherData
identifier	airTemperatures groundTemperatures windSpeeds windDirections pressures rainfall
reportWeather () reportStatus () powerSave (instruments) remoteControl (commands) reconfigure (commands) restart (instruments) shutdown (instruments)	collect () summarize ()

Ground thermometer	Anemometer	Barometer
gt_Ident temperature	an_Ident windSpeed windDirection	bar_Ident pressure height
get () test ()	get () test ()	get () test ()

7.1.4. Design models

- Design models show the objects and object classes and relationships between these entities.
- Static models describe the static structure of the system in terms of object classes and relationships.
- Dynamic models describe the dynamic interactions between objects.

7.1.4 Examples of design models (continued)

- **Subsystem models** that show logical groupings of objects into coherent subsystems.
- **Sequence models** that show the sequence of object interactions.
- **State machine models** that show how individual objects change their state in response to events.
- Other models include use-case models, aggregation models, generalisation models, etc.

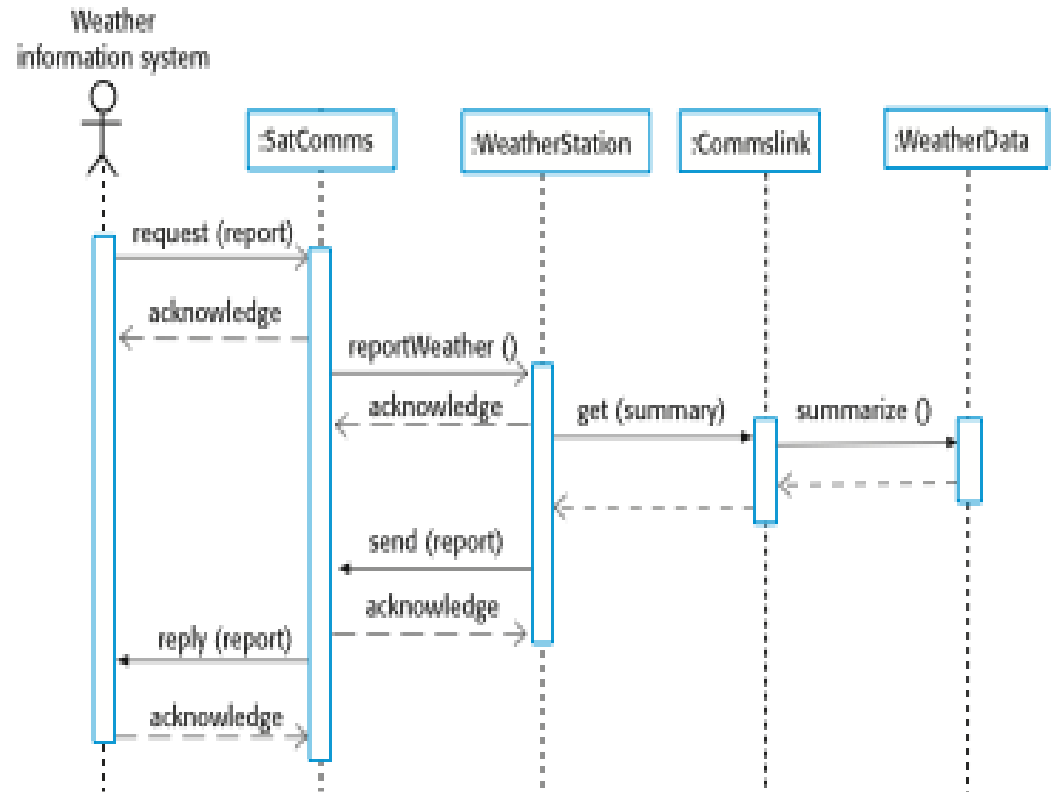
7.1.4 Subsystem models (continued)

- Shows how the design is organised into logically related groups of objects.
- In the UML, these are shown using packages - an encapsulation construct. This is a logical model. The actual organisation of objects in the system may be different.

7.1.4 Sequence diagram describing data collection (continued)

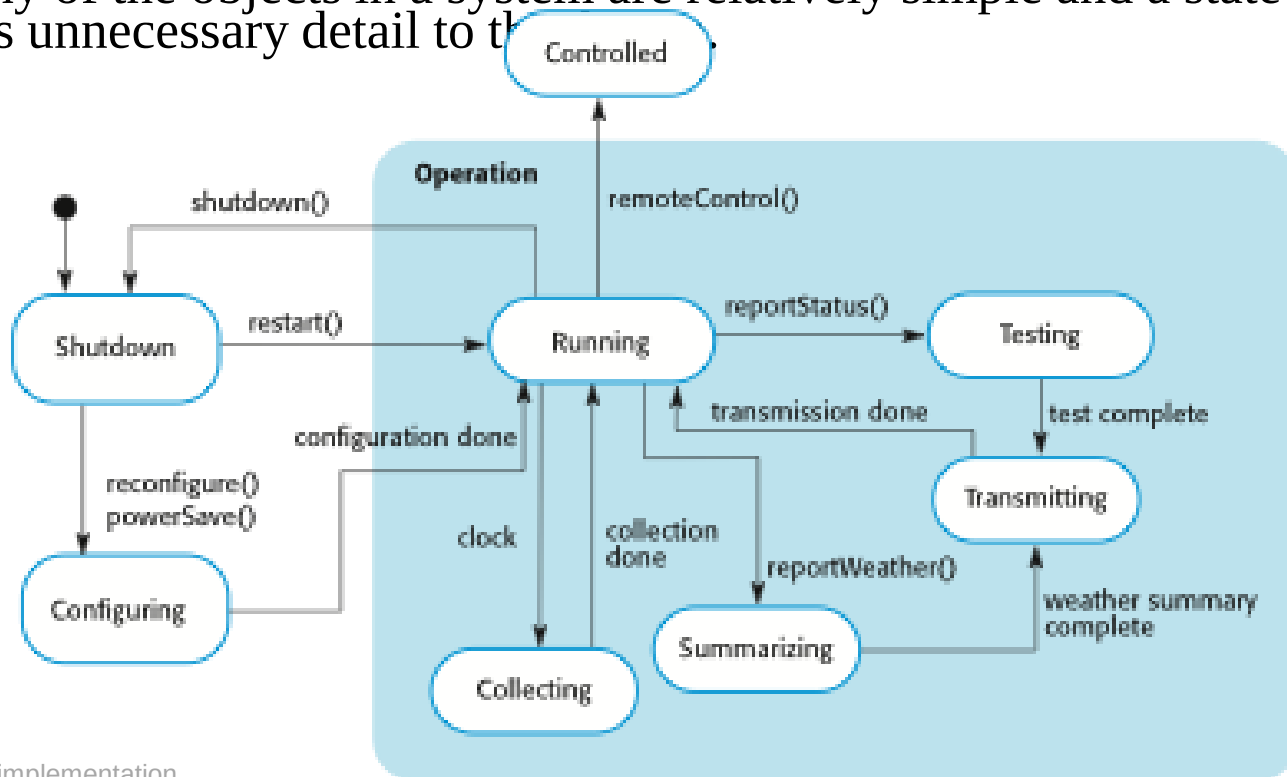
Sequence models show the sequence of object interactions that take place

- Objects are arranged horizontally across the top;
- Time is represented vertically so models are read top to bottom;
- Interactions are represented by labelled arrows. Different styles of arrow represent different types of interaction;
- A thin rectangle in an object lifeline represents the time when the object is the controlling object in the system.



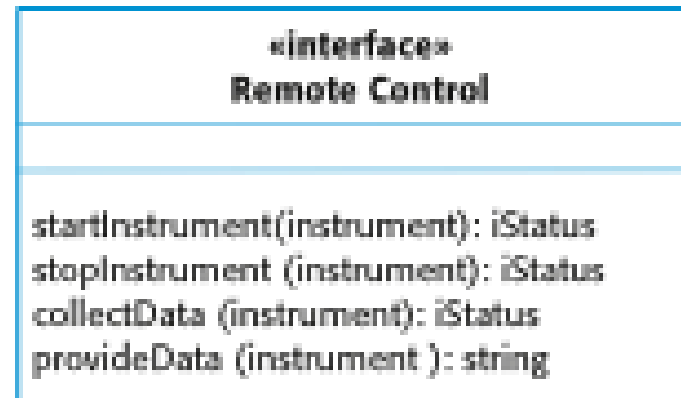
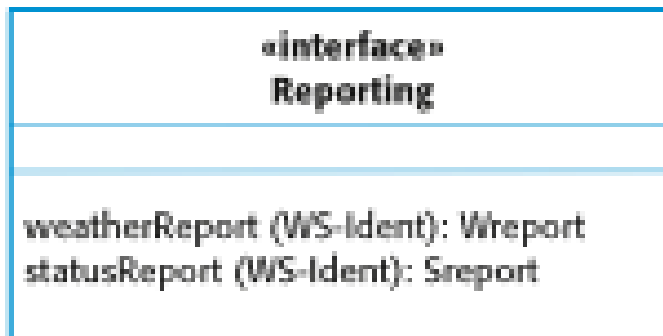
7.1.4 State diagrams (continued)

- State diagrams are used to show how objects respond to different service requests and the state transitions triggered by these requests.
- State diagrams are useful high-level models of a system or an object's run-time behavior.
- You don't usually need a state diagram for all of the objects in the system. Many of the objects in a system are relatively simple and a state model adds unnecessary detail to the



7.1.5 Interface specification

- Object interfaces have to be specified so that the objects and other components can be designed in parallel.
- Designers should avoid designing the interface representation but should hide this in the object itself.
- Objects may have several interfaces which are viewpoints on the methods provided.
- The UML uses class diagrams for interface specification but Java may also be used.



7.2 Design patterns

- A design pattern is a way of reusing abstract knowledge about a problem and its solution.
- A pattern is a description of the problem and the essence of its solution.
- It should be sufficiently abstract to be reused in different settings.
- Pattern descriptions usually make use of object-oriented characteristics such as inheritance and polymorphism.

7.2 Pattern elements (continued)

- Name
 - A meaningful pattern identifier.
- Problem description.
- Solution description.
 - Not a concrete design but a template for a design solution that can be instantiated in different ways.
- Consequences
 - The results and trade-offs of applying the pattern.

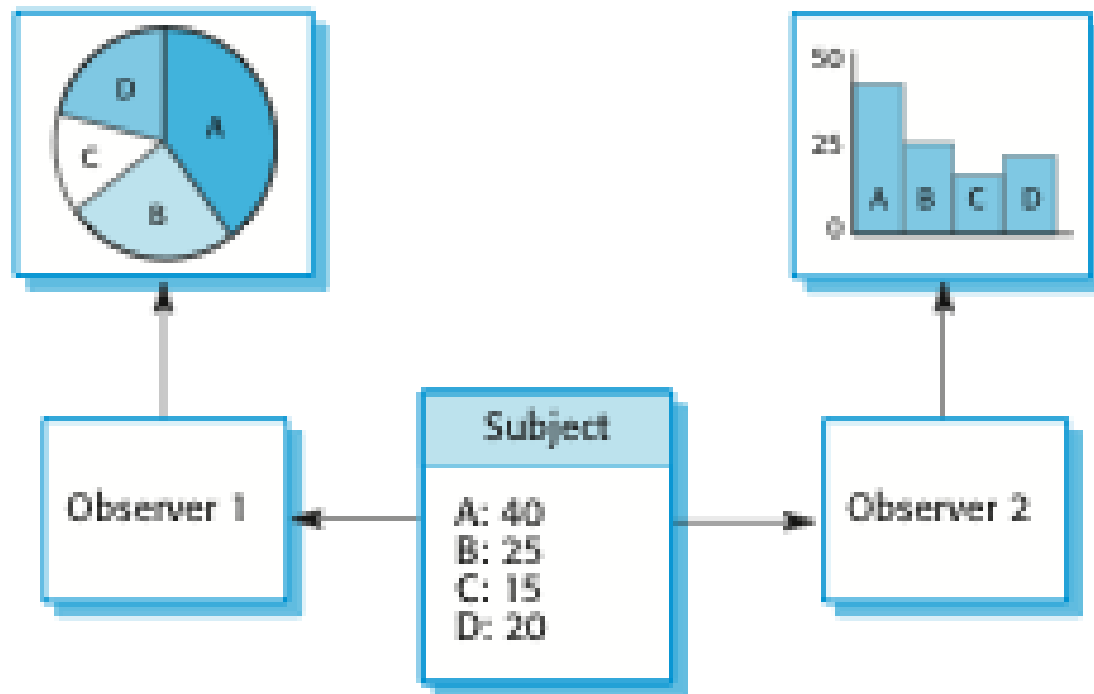
An Observer pattern

- ✧ Name
 - **Observer.**
- ✧ Description
 - Separates the display of object state from the object itself.
- ✧ Problem description
 - Used when multiple displays of state are needed.
- ✧ Solution description
 - See slide with UML description.
- ✧ Consequences
 - Optimisations to enhance display performance are impractical.

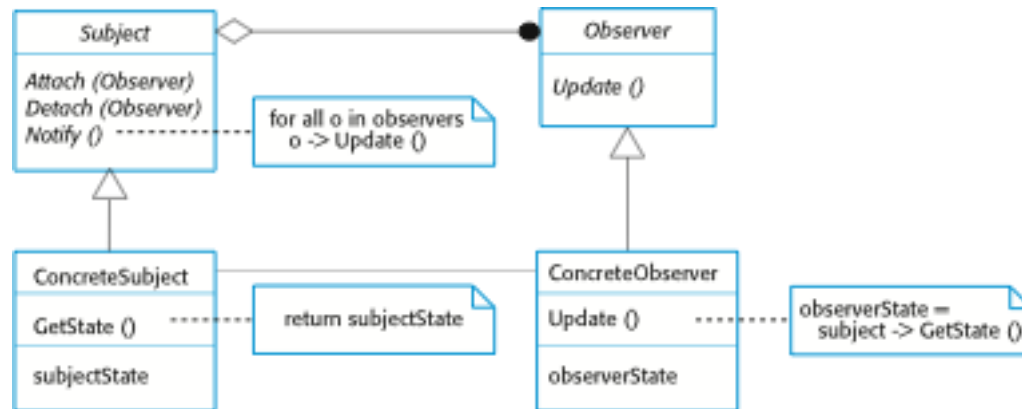
7.2 The Observer pattern (continued)

Pattern name	Observer
Description	Separates the display of the state of an object from the object itself and allows alternative displays to be provided. When the object state changes, all displays are automatically notified and updated to reflect the change.
Problem description	In many situations, you have to provide multiple displays of state information, such as a graphical display and a tabular display. This pattern may be used in all situations where more than one display format for state information is required and where it is not necessary for the object that maintains the state information to know about the specific display formats used.
Solution description	This involves two abstract objects, Subject and Observer, and two concrete objects, ConcreteSubject and ConcreteObject, which inherit the attributes of the related abstract objects. The abstract objects include general operations that are applicable in all situations. The state to be displayed is maintained in ConcreteSubject, which inherits operations from Subject allowing it to add and remove Observers (each observer corresponds to a display) and to issue a notification when the state has changed. The ConcreteObserver maintains a copy of the state of ConcreteSubject and implements the Update() interface of Observer that allows these copies to be kept in step.
Consequences	The subject only knows the abstract Observer and does not know details of the concrete class. Therefore there is minimal coupling between these objects.

7.2 Multiple displays using the Observer pattern (continued)



7.2 A UML model of the Observer pattern (continued)



7.2 Design problems (continued)

- To use patterns in your design, you need to recognize that any design problem you are facing may have an associated pattern that can be applied.
 - Tell several objects that the state of some other object has changed (Observer pattern).
 - Tidy up the interfaces to a number of related objects that have often been developed incrementally (Façade pattern).
 - Provide a standard way of accessing the elements in a collection, irrespective of how that collection is implemented (Iterator pattern).
 - Allow for the possibility of extending the functionality of an existing class at run-time (Decorator pattern).

7.3 Implementation issues

- Focus here is not on programming, although this is obviously important, but on other implementation issues that are often not covered in programming texts:
 - **Reuse** Most modern software is constructed by reusing existing components or systems. When you are developing software, you should make as much use as possible of existing code.
 - **Configuration management** During the development process, you have to keep track of the many different versions of each software component in a configuration management system.
 - **Host-target development** Production software does not usually execute on the same computer as the software development environment. Rather, you develop it on one computer (the host system) and execute it on a separate computer (the target system).

7.3.1 Reuse

- From the 1960s to the 1990s, most new software was developed from scratch, by writing all code in a high-level programming language.
 - The only significant reuse of software was the reuse of functions and objects in programming language libraries.
- Costs and schedule pressure mean that this approach became increasingly unviable, especially for commercial and Internet-based systems.
- An approach to development based around the reuse of existing software emerged and is now generally used for business and scientific software.

7.3.1 Reuse levels

- The abstraction level
 - At this level, you don't reuse software directly but use knowledge of successful abstractions in the design of your software.
- The object level
 - At this level, you directly reuse objects from a library rather than writing the code yourself.
- The component level
 - Components are collections of objects and object classes that you reuse in application systems.
- The system level
 - At this level, you reuse entire application systems.

7.3.1 Reuse costs

- The costs of the time spent in looking for software to reuse and assessing whether or not it meets your needs.
- Where applicable, the costs of buying the reusable software. For large off-the-shelf systems, these costs can be very high.
- The costs of adapting and configuring the reusable software components or systems to reflect the requirements of the system that you are developing.
- The costs of integrating reusable software elements with each other (if you are using software from different sources) and with the new code that you have developed.

7.3.2 Configuration management

- Configuration management is the name given to the general process of managing a changing software system.
- The aim of configuration management is to support the system integration process so that all developers can access the project code and documents in a controlled way, find out what changes have been made, and compile and link components to create a system.
- See also Chapter 25.

7.3.2 Configuration management activities

- Version management, where support is provided to keep track of the different versions of software components. Version management systems include facilities to coordinate development by several programmers.
- System integration, where support is provided to help developers define what versions of components are used to create each version of a system. This description is then used to build a system automatically by compiling and linking the required components.
- Problem tracking, where support is provided to allow users to report bugs and other problems, and to allow all developers to see who is working on these problems and when they are fixed.

7.3.3 Host-target development

- Most software is developed on one computer (the host), but runs on a separate machine (the target).
- More generally, we can talk about a development platform and an execution platform.
 - A platform is more than just hardware.
 - It includes the installed operating system plus other supporting software such as a database management system or, for development platforms, an interactive development environment.
- Development platform usually has different installed software than execution platform; these platforms may have different architectures.

7.3.3 Development platform tools

- An integrated compiler and syntax-directed editing system that allows you to create, edit and compile code.
- A language debugging system.
- Graphical editing tools, such as tools to edit UML models.
- Testing tools, such as Junit that can automatically run a set of tests on a new version of a program.
- Project support tools that help you organize the code for different development projects.

7.3.3 Integrated development environments (IDEs)

- Software development tools are often grouped to create an integrated development environment (IDE).
- An IDE is a set of software tools that supports different aspects of software development, within some common framework and user interface.
- IDEs are created to support development in a specific programming language such as Java. The language IDE may be developed specially, or may be an instantiation of a general-purpose IDE, with specific language-support tools.

Design patterns(Cont..)

- Design patterns represent the best practices used by experienced object-oriented software developers. Design patterns are solutions to general problems that software developers faced during software development. These solutions were obtained by trial and error by numerous software developers over quite a substantial period of time

What is Gang of Four (GOF)?

- In 1994, four authors Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides published a book titled **Design Patterns - Elements of Reusable Object-Oriented Software** which initiated the concept of Design Pattern in Software development.
- These authors are collectively known as **Gang of Four (GOF)**. According to these authors design patterns are primarily based on the following principles of object orientated design.
- Program to an interface not an implementation
- Favor object composition over inheritance

Usage of Design Pattern

Design Patterns have two main usages in software development.

- Common platform for developers
- Best Practices

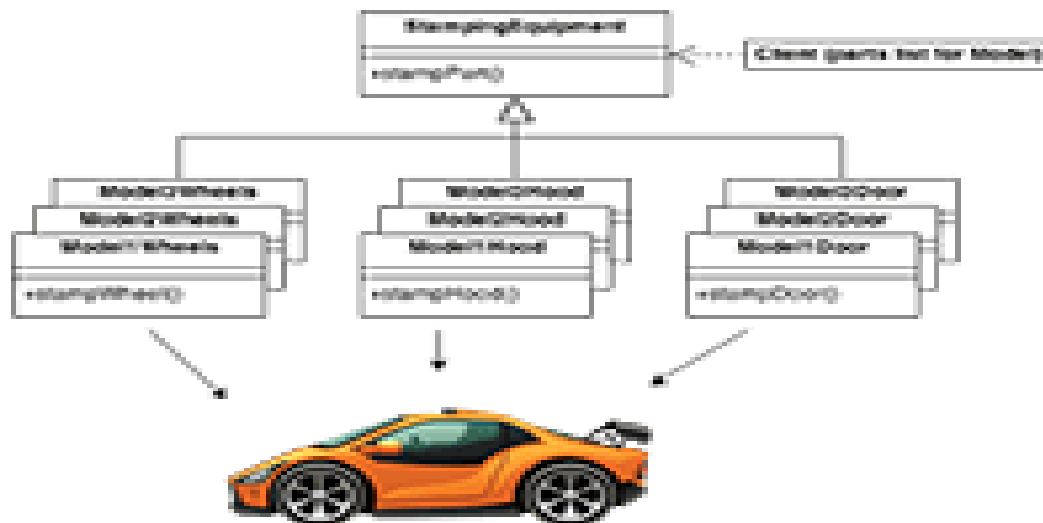
Types of Design Patterns

- As per the design pattern reference book **Design Patterns - Elements of Reusable Object-Oriented Software** , there are 23 design patterns which can be classified in three categories:
- Creational
- Structural
- and Behavioral patterns.

S.N.	Pattern & Description
1	Creational Patterns These design patterns provide a way to create objects while hiding the creation logic, rather than instantiating objects directly using new operator. This gives program more flexibility in deciding which objects need to be created for a given use case.
2	Structural Patterns These design patterns concern class and object composition. Concept of inheritance is used to compose interfaces and define ways to compose objects to obtain new functionalities.
3	Behavioral Patterns These design patterns are specifically concerned with communication between objects.

Creational design patterns

- These design patterns are all about class instantiation. This pattern can be further divided into class-creation patterns and object-creational patterns. While class-creation patterns use inheritance effectively in the instantiation process, object-creation patterns use delegation effectively to get the job done.



➤ **Abstract Factory**

Creates an instance of several families of classes

➤ **Builder**

Separates object construction from its representation

➤ **Factory Method**

Creates an instance of several derived classes

➤ **Object Pool**

Avoid expensive acquisition and release of resources by recycling objects that are no longer in use

➤ **Prototype**

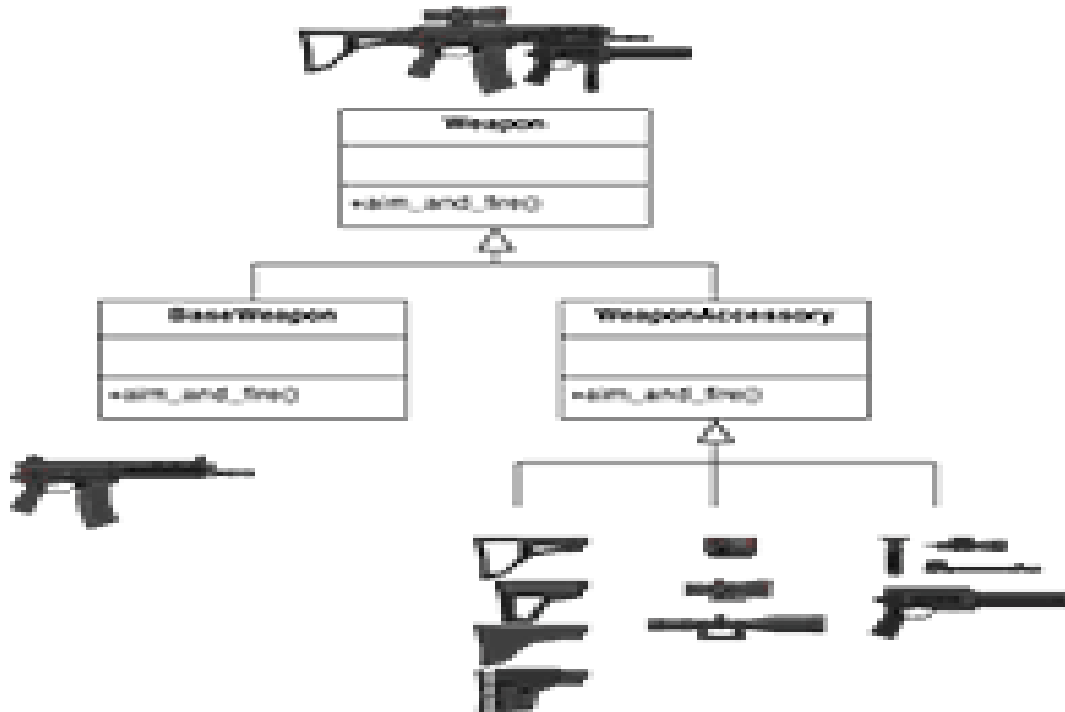
A fully initialized instance to be copied or cloned

➤ **Singleton**

A class of which only a single instance can exist

Structural design patterns

- These design patterns are all about Class and Object composition. Structural class-creation patterns use inheritance to compose interfaces. Structural object-patterns define ways to compose objects to obtain new functionality.



- **Adapter**
Match interfaces of different classes
- **Bridge**
Separates an object's interface from its implementation
- **Composite**
A tree structure of simple and composite objects
- **Decorator**
Add responsibilities to objects dynamically
- **Facade**
A single class that represents an entire subsystem
- **Flyweight**
A fine-grained instance used for efficient sharing

➤ **Private Class Data**

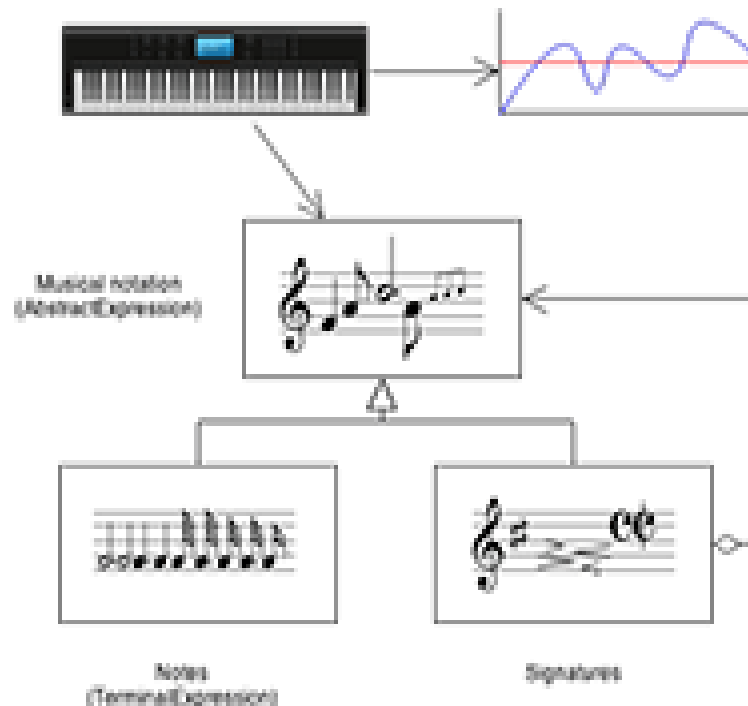
Restricts accessor/mutator access

➤ **Proxy**

An object representing another object

Behavioral design patterns

- These design patterns are all about Class's objects communication. Behavioral patterns are those patterns that are most specifically concerned with communication between objects.



➤ **Chain of responsibility**

A way of passing a request between a chain of objects

➤ **Command**

Encapsulate a command request as an object

➤ **Interpreter**

A way to include language elements in a program

➤ **Iterator**

Sequentially access the elements of a collection

➤ **Mediator**

Defines simplified communication between classes

➤ **Memento**

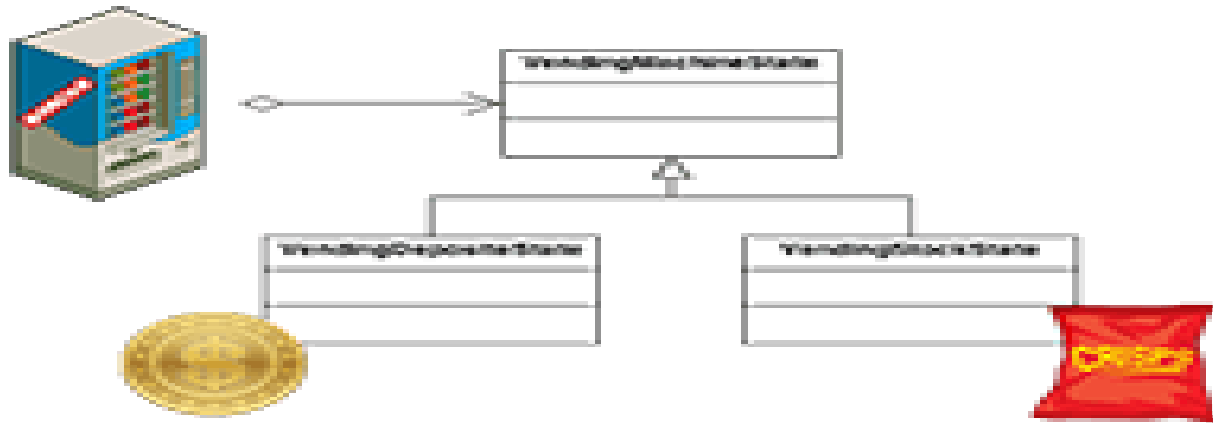
Capture and restore an object's internal state

➤ **Null Object**

Designed to act as a default value of an object

➤ **Observer**

A way of notifying change to a number of classes



- State
Alter an object's behavior when its state changes
- Strategy
Encapsulates an algorithm inside a class

➤ **Template method**

Defer the exact steps of an algorithm to a subclass

➤ **Visitor**

Defines a new operation to a class without change